

stage2_cli_agents_catalog

HelixAgent CLI Agents Master Catalog

Generated from: HelixDevelopment/HelixAgent cli_agents/ directory Total
Markdown Analysis Files: 22 Total Git Submodule Agents: 62 Total Agents
Cataloged: 60+ unique CLI agents Analysis Date: 2026-04-04

Table of Contents

1. [Project Overview](#)
 2. [Complete Agent Inventory](#)
 3. [Tier 1 Agent Deep Dives](#)
 4. [Tier 2 Agent Deep Dives](#)
 5. [Tier 3-5 Agent Summaries](#)
 6. [Feature Matrix](#)
 7. [Master Integration Plan](#)
 8. [Porting Priorities](#)
 9. [Key Innovations & Genial Hacks](#)
 10. [Submodule Repository Map](#)
-

1. Project Overview

HelixAgent's cli_agents/ directory contains a comprehensive competitive analysis of 60+ CLI coding agents. Each agent is represented as either: - **Git submodule** (actual source code cloned for deep analysis) - **Markdown analysis document** (feature extraction and porting recommendations)

Analysis Methodology

- Every agent was cloned via git submodule for line-of-code analysis
- Key files were measured and documented
- Feature matrices were created for cross-agent comparison
- Porting recommendations were assigned priority levels

Source Repository

- **URL:** <https://github.com/HelixDevelopment/HelixAgent>
 - **Directory:** cli_agents/
 - **License:** Mixed (varies by submodule)
-

2. Complete Agent Inventory

Tier 1: Production-Ready (10 agents)

#	Agent	Source	Language	LOC	License	Analysis File
1	claude-code-source	Anthropic internal	TypeScript	513K	Proprietary	TIER_1_SUMMARY.md
2	aider	Aider-AI/ aider	Python	~20K	Apache 2.0	AIDER_ANALYSIS.md
3	codex	openai/ codex	TS/Rust	~10K	Apache 2.0	CODEX_ANALYSIS.md
4	openhands	All-Hands-AI/ OpenHands	Python	~50K	MIT	OPENHANDS_ANALYSIS.md
5	cline	cline/cline	TypeScript	~15K	Apache 2.0	CLINE_ANALYSIS.md
6	continue	continuedev/ continue	TypeScript	~30K	Apache 2.0	CONTINUE_ANALYSIS.md
7	gemini-cli	google-gemini/ gemini-cli	TypeScript	~5K	Apache 2.0	GEMINI_CLI_ANALYSIS.md
8	roo-code	RooVetGit/ Roo-Code	TypeScript	~15K	Apache 2.0	ROO_CODE_ANALYSIS.md
9	swe-agent	princeton-nlp/SWE-agent	Python	~10K	MIT	SWE_AGENT_ANALYSIS.md
10	vtcode	vinhnx/ vtcode	Swift	~2K	MIT	VTCODE_ANALYSIS.md

Tier 2: Emerging/Notable (9 agents)

#	Agent	Source	Language	License	Analysis File
11	gptme	ErikBjare/ gptme	Python	MIT	GPTME_ANALYSIS.md
12	gpt-engineer	AntonOsika/ gpt-engineer	Python	MIT	GPT_ENGINEER_ANALYSIS.md
13	kilo-code	Kilo-Org/ kilocode	TypeScript	Apache 2.0	KILO_CODE_ANALYSIS.md
14	junie	JetBrains/junie	Kotlin/ Java	Proprietary	JUNIE_ANALYSIS.md
15	forge	antinomyhq/ forge	Rust	Apache 2.0	FORGE_ANALYSIS.md
16			Python	MIT	DEEPSEEK_CLI_ANALYSIS.md

#	Agent	Source	Language	License	Analysis File
	deepseek-cli	holasoymalva/deepseek-cli			
17	agent-deck	asheshgoplani/agent-deck	TypeScript	MIT	AGENT_DECK_ANALYSIS.md
18	zeroshot	supercrafter100/zeroshot	TypeScript	MIT	ZEROSHOT_ANALYSIS.md
19	spec-kit	github/spec-kit	Ruby	MIT	SPEC_KIT_ANALYSIS.md

Tier 3: Notable/Niche (10 agents)

#	Agent	Source	Language	Key Feature
20	amazon-q	aws/amazon-q-developer-cli	Rust	AWS integration, cloud-native
21	claude-plugins	(plugin system)	TypeScript	Plugin architecture
22	claude-squad	(multi-agent)	TypeScript	Agent swarms
23	open-interpreter	KillianLucas/open-interpreter	Python	Code execution, data analysis
24	plandex	plandex-ai/plandex	Go	Task decomposition, planning
25	ollama-code	tcsenpai/ollama-code	Python	Local model support
26	qwen-code	QwenLM/qwen-code	Python	Qwen model support
27	codai	meysamhadeli/codai	Python	AI coding assistant
28	codename-goose	jgenerali/codename-goose	Python	Agent framework
29	codex-skills	openai/codex-skills	TypeScript	Skill system

Tier 4: Specialized (10 agents)

#	Agent	Source	Language	Key Feature
30	git-mcp	(MCP server)	TypeScript	Git via Model Context Protocol
31	postgres-mcp	(MCP server)	TypeScript	PostgreSQL MCP operations
32	mistral-code	(Mistral AI)	Python	Mistral model support
33	octogen	(code gen)	Python	Multi-agent code generation

#	Agent	Source	Language	Key Feature
34	nanocoder	(lightweight)	TypeScript	Lightweight coding
35	taskweaver	microsoft/ TaskWeaver	Python	Data analytics focus
36	opencode-cli	opencode-ai/ opencode	TypeScript	VS Code extension + CLI
37	fauxpilot	(Copilot alt)	Python	Self-hosted completion
38	kiro-cli	(broken remote)	Unknown	Unknown
39	conduit	(unknown)	Unknown	Workflow tool

Tier 5: Experimental/Minimal (15+ agents)

#	Agent	Source	Language	Notes
40	aichat	(unknown)	Unknown	Chat agent
41	aiagent	(unknown)	Unknown	Generic agent
42	aichat-llm-functions	(unknown)	Unknown	LLM functions
43	copilot-cli	github/copilot-cli	Unknown	GitHub Copilot CLI
44	crush	(unknown)	Unknown	Unknown
45	deepseek-cli-youkpan	(alt impl)	Python	Alternative DeepSeek CLI
46	get-shit-done	(unknown)	Unknown	Productivity tool
47	mobile-agent	(unknown)	Unknown	Mobile-focused
48	multiagent-coding	(unknown)	Unknown	Multi-agent
49	noi	(unknown)	Unknown	Unknown
50	shai	(unknown)	Unknown	Unknown
51	snow-cli	(unknown)	Unknown	Unknown
52	superset	(unknown)	Unknown	Unknown
53	ui-ux-pro-max	(unknown)	Unknown	UI/UX focused
54	warp	(terminal)	Unknown	Terminal integration
55	x-cmd	(unknown)	Unknown	Command tool
56	xela-cli	(unknown)	Unknown	Unknown

3. Tier 1 Agent Deep Dives

3.1 AIDER (Aider-AI/aider) - Git-Native Pair Programming

Stats: 5.7M+ PyPI installs, 15B+ tokens/week, 88% self-written code

Feature Inventory

Commands (Slash Commands) - `commands.py` (1712 lines): | Command | Purpose | |-----|
|-----| | `/add` | Add files to context | | `/drop` | Remove files from context | | `/commit` | Commit changes | | `/diff` | Show diffs | | `/undo` | Undo last change | | `/lint` | Run linters | | `/test` | Run tests | | `/help` | Show help | | `/voice` | Voice input | | `/model` | Switch models | | `/settings` | Configure settings |

Core Architecture: - `main.py` (1274 lines) - Entry point - `models.py` (1323 lines) - LLM abstraction layer (Claude, GPT-4o, o1, o3-mini, DeepSeek, Ollama, 100+ via OpenRouter) - `repo.py` (622 lines) - Git integration (auto-commit, branch management, diff, conflict resolution) - `repomap.py` (867 lines) - Repository mapping with tree-sitter (100+ languages) - `io.py` (1191 lines) - I/O handling - `linter.py` (304 lines) - Linting & testing with error-fix loops - `voice.py` (187 lines) - Speech-to-code - `scrape.py` (284 lines) - URL/web content extraction - `watch.py` (318 lines) - File watching & IDE integration

Coder Modes (`coders/` ~3000 lines): - **EditBlock Coder:** Surgical code edits - **Architect Coder:** High-level design - **Ask Coder:** Q&A mode - **Context Coder:** Context-aware coding - **Patch Coder:** Diff-based editing - **Search/Replace:** Direct text replacement

Power Features & Innovations

- 1. **Repository Mapping** - Tree-sitter based codebase analysis with intelligent context selection
- 2. **Edit Block Format** - Surgical search/replace code modifications with minimal diff generation
- 3. **Git-Native Workflow** - Automatic commits with sensible messages, branch-aware operations
- 4. **Voice-to-Code** - Full speech recognition integration for hands-free coding
- 5. **Lint/Test Auto-Fix Loops** - Automatic quality checks with iterative error fixing
- 6. **Web Scraping** - URL content extraction for documentation context
- 7. **File Watching** - IDE integration with comment-driven development

APIs & Integration Points

Aider Feature	HelixAgent Implementation
Repository Map	Extend ToolSchema with TreeView, Symbols
Edit Blocks	ToolEdit with search/replace
Git Integration	ToolGit with auto-commit
Voice Commands	KAIROS voice mode extension
Lint/Test	ToolLint, ToolTest integration
Slash Commands	CLI command system

Porting Priority: HIGH

3.2 CODEX (openai/codex) - OpenAI Official, Sandboxed

Architecture: TypeScript CLI frontend + Rust core implementation

Feature Inventory

Core Components: - `codex-cli/` - TypeScript CLI frontend (~2000 lines) - `codex-rs/core` - Core Rust implementation (~5000 lines) - `codex-rs/tui` - Terminal UI with ratatui (~3000 lines) - `app-server-protocol/` - Protocol definitions - `sdk/` - SDK for integrations

Key Features: 1. **Sandboxed Execution** - macOS Seatbelt (`/usr/bin/sandbox-exec`), network-disabled by default 2. **Multi-Modal Input** - Natural language, file attachments, image context 3. **Approval System** - Configurable approval policies (command execution, patch application) 4. **Git Integration** - Automatic commits, branch management, diff viewing 5. **Collaboration Modes** - Full autonomy, interactive, approval-required 6. **TUI (Terminal UI)** - ratatui-based rich interactive interface with chat-style interaction 7. **Protocol-Based Communication** - JSON-RPC Lite protocol, structured request/response, thread history, conversation summaries

Security Features

- 1. Sandboxing: All commands run in sandbox
- 2. Network Control: Can disable network access
- 3. Approval Gates: Multi-level approval system
- 4. Audit Logging: Complete operation logs

APIs & Integration Points

Codex Feature	HelixAgent Implementation
Sandboxed Execution	ToolExecutor with permission modes
Approval System	PermissionMode + rules
TUI	Web UI + Terminal UI
Git Integration	ToolGit with auto-commit
Protocol	API endpoints
Sandboxing	Container-based isolation

Porting Priority: HIGH

3.3 CLINE (cline/cline) - Browser Automation, VS Code

Type: VS Code extension + CLI capabilities

Feature Inventory

- 1. **Autonomous Task Execution** - Plans complex tasks, executes multi-step workflows, self-corrects
- 2. **Browser Automation** - Headless browser, web navigation, screenshot analysis, API documentation reading
- 3. **Multi-Provider Support** - OpenAI, Anthropic, Google Gemini, Ollama local models
- 4. **File Context Management** - Add/remove files, intelligent selection, gitignore-aware
- 5. **Terminal Integration** - Shell command execution, output reading, monitoring
- 6. **Diff-Based Editing** - Precise code modifications, search/replace blocks, minimal change principle

Architecture

```
cline/  
├── src/  
│   ├── core/           # Core agent logic  
│   ├── services/       # LLM services  
│   ├── integrations/   # Browser, terminal  
│   └── shared/         # Shared utilities  
└── extension.ts        # VS Code extension entry
```

APIs & Integration Points

Cline Feature	HelixAgent Implementation
Autonomous Execution	SubAgent system
Browser	Browser automation tools
Terminal	ToolBash integration
Diff Editing	ToolEdit with blocks
Context	File management

Porting Priority: HIGH

3.4 OPENHANDS (All-Hands-AI/OpenHands) - Autonomous SWE

Stats: SWE-bench score 77.6%, multi-language support

Feature Inventory

Product Editions: 1. **OpenHands SDK** - Composable Python library, scalable from local to cloud 2. **OpenHands CLI** - Familiar CLI (like Claude Code/Codex) 3. **OpenHands Local GUI** - REST API + React SPA, Devin/Jules-like experience 4. **OpenHands Cloud** - Hosted infrastructure, free tier with Minimax model 5. **OpenHands Enterprise** - Self-hosted VPC, Kubernetes, RBAC, Slack/Jira/Linear integrations

Core Architecture:

```
openhands/  
├── openhands/           # Core Python package
```

— agenthub/	# Agent implementations
— controller/	# Agent controller
— events/	# Event system
— runtime/	# Runtime environment
— server/	# REST API server
— frontend/	# React frontend
— evaluation/	# Evaluation framework
— enterprise/	# Enterprise features

Key Features: 1. **Agent System** - Multiple implementations, AgentHub for management 2. **Runtime Environment** - Docker-based sandboxing, local/remote/E2B support 3. **Event-Driven Architecture** - Event system for actions, streamable observations, action-observation loop 4. **Evaluation Framework** - SWE-bench integration, custom metrics, benchmark infrastructure 5. **Multi-Modal Support** - Browser automation, image understanding, screenshots 6. **Theory of Mind Module** - Research project for agent cognition

Enterprise Features: - Slack, Jira, Linear integrations - Team collaboration - RBAC (Role-based access control) - Kubernetes deployment - Source-available licensing

APIs & Integration Points

OpenHands Feature	HelixAgent Implementation
Agent System	SubAgent system
Runtime	Container adapter
Event System	EventBus module
Evaluation	Benchmark module
Browser	Browser automation tools
Multi-user	Team management

Porting Priority: HIGH

3.5 CONTINUE (continuedev/continue) - Universal IDE

Type: VS Code extension + JetBrains plugin + any IDE with LSP

Feature Inventory

1. **Universal IDE Support** - VS Code, JetBrains, any IDE with LSP
2. **Code Completion** - Tab-based, multi-line, context-aware
3. **Chat Interface** - Inline chat, sidebar chat, code explanations
4. **Multi-Provider Support** - OpenAI, Anthropic, Ollama, Custom APIs
5. **Custom Context Providers:**
 - @file - File context
 - @url - Web page context
 - @docs - Documentation
 - @codebase - Codebase search
6. **Actions (Slash Commands):**
 - /edit - Edit code
 - /comment - Add comments

- `/doc` - Generate documentation
- `/test` - Generate tests

Architecture

```

continue/
├── core/           # Core logic
├── extensions/     # IDE extensions
│   ├── vscode/
│   └── intellij/
├── gui/           # Chat interface
└── binary/        # CLI binary
  
```

APIs & Integration Points

Continue Feature	HelixAgent Implementation
Context Providers	Tool system
Actions	CLI commands
Chat	Conversation context
Completion	Code completion API

Porting Priority: HIGH

3.6 ROO CODE (RooVetGit/Roo-Code) - Multi-File Editing

Formerly: Roo Cline (fork with advanced capabilities)

Feature Inventory

1. **Multi-File Editing** - Edit multiple files simultaneously, cross-file refactoring, consistent codebase changes
2. **Context Management** - Smart context window, automatic compression, token optimization
3. **Agent Modes:**
 - Code mode
 - Architect mode
 - Ask mode
 - Debug mode
4. **Tool Use** - File read/write, terminal execution, web search, code search
5. **Multi-Provider Support** - OpenAI, Anthropic, Google, Local models

APIs & Integration Points

Roo Code Feature	HelixAgent Implementation
Multi-file edits	Batch edit operations
Context mgmt	Context window optimization
Agent modes	Agent type system

Roo Code Feature	HelixAgent Implementation
Tools	ToolExecutor

Porting Priority: HIGH

3.7 GEMINI CLI (google-gemini/gemini-cli) - Google Official

Type: Official Google CLI for Gemini models

Feature Inventory

1. **Interactive Chat** - Conversational, multi-turn, context preservation
2. **File Operations** - Read, write, file context in chat
3. **Gemini Model Access** - Direct access, official API, latest updates
4. **Streaming Responses** - Real-time output, streaming tokens, progress indication
5. **Command Integration** - Execute shell commands, command output as context, tool use

APIs & Integration Points

Gemini CLI Feature	HelixAgent Implementation
Chat	Conversation handlers
Files	ToolRead/ToolWrite
Commands	ToolBash
Streaming	SSE/streaming endpoints

Porting Priority: MEDIUM

3.8 SWE-AGENT (princeton-nlp/SWE-agent) - SWE-bench Focused

Stats: Princeton NLP research project

Feature Inventory

1. **SWE-bench Integration** - Evaluated on benchmark, real GitHub issue resolution, bug fixing
2. **Computer Interface** - Filesystem navigation, file viewing, file editing, command execution
3. **Thought-Action Loop** - Reasoning before action, execution, observation feedback, iterative improvement
4. **Demonstration Trajectories** - Training on demonstrations, few-shot learning, trajectory replay
5. **Multi-Model Support** - GPT-4, Claude, Open-source models

APIs & Integration Points

SWE-agent Feature	HelixAgent Implementation
Thought-Action	Agent workflow
Computer Interface	Tool system
Environment	Container runtime
Evaluation	Benchmark system

Porting Priority: HIGH

3.9 VTCODE (vinhnx/vtcode) - Minimal Swift

Type: Lightweight CLI coding assistant

Feature Inventory

1. **File Operations** - Read, write, list directories
2. **Chat Interface** - Natural language queries, code explanations, simple modifications
3. **Swift Implementation** - Native Swift, macOS optimized, fast execution

APIs & Integration Points

VTCode Feature	HelixAgent Implementation
File ops	Tool system
Chat	Conversation handlers

Porting Priority: LOW

4. Tier 2 Agent Deep Dives

4.1 GPTME (ErikBjare/gptme) - Personal AI Assistant

Language: Python | **License:** MIT

Key Features: - Local-first approach - Privacy focused - Python tool ecosystem - Jupyter notebook integration - Self-improvement capabilities

Porting Priority: MEDIUM

4.2 GPT-ENGINEER (AntonOsika/gpt-engineer) - Prompt-to-Code

Language: Python | **License:** MIT

Key Features: - Prompt-to-code generation - Iterative refinement - Identity selection (coding styles) - Multi-file code generation

Porting Priority: MEDIUM

4.3 KILO CODE (Kilo-Org/kilocode) - Cline Fork

Language: TypeScript | **License:** Apache 2.0

Key Features: - Based on Cline - VS Code extension - Additional model support - Enhanced UI

Porting Priority: MEDIUM (features overlap with Cline)

4.4 JUNIE (JetBrains/junie) - JetBrains IDE Assistant

Language: Kotlin/Java | **License:** Proprietary

Key Features: - JetBrains IDE integration - Code completion - Chat interface - Multi-language support

Porting Priority: LOW (proprietary, IDE-specific)

4.5 FORGE (antinomyhq/forge) - Next-Gen Rust Agent

Language: Rust | **License:** Apache 2.0

Key Features: - Rust implementation - High performance - CLI-first design - Modern architecture

Porting Priority: MEDIUM

4.6 DEEPSEEK CLI (holasoymlva/deepseek-cli) - DeepSeek Access

Language: Python | **License:** MIT

Key Features: - DeepSeek model access - Simple CLI interface - Chat mode - Code generation

Porting Priority: LOW (HelixAgent already supports DeepSeek)

4.7 AGENT DECK (asheshgoplani/agent-deck) - Agent Framework

Language: TypeScript | **License:** MIT

Key Features: - Agent orchestration - Workflow management - Multi-agent support

Porting Priority: MEDIUM

4.8 ZEROSHOT (supercrafter100/zeroshot) - Zero-Shot Coding

Language: TypeScript | **License:** MIT

Key Features: - Zero-shot generation - Task automation - CLI interface

Porting Priority: LOW

4.9 SPEC KIT (github/spec-kit) - GitHub Specification Toolkit

Language: Ruby | **License:** MIT

Key Features: - Specification management - GitHub integration - Structured data handling

Porting Priority: LOW

5. Tier 3-5 Agent Summaries

Tier 3: Notable/Niche (10 agents)

Agent	Language	Key Differentiator
amazon-q	Rust	AWS integration, cloud-native
claude-plugins	TypeScript	Plugin architecture, extensibility
claude-squad	TypeScript	Agent swarms, team coordination
open-interpreter	Python	Code execution, data analysis
plandex	Go	Task decomposition, planning
ollama-code	Python	Local model support
qwen-code	Python	Qwen model support
codai	Python	AI coding assistant
codename-goose	Python	Agent framework
codex-skills	TypeScript	Skill system

Tier 4: Specialized (10 agents)

Agent	Language	Key Differentiator
git-mcp	TypeScript	Git via MCP
postgres-mcp	TypeScript	PostgreSQL via MCP
mistral-code	Python	Mistral model support
octogen	Python	Multi-agent code generation
nanocoder	TypeScript	Lightweight coding
taskweaver	Python	Data analytics focus
opencode-cli	TypeScript	VS Code + CLI
fauxpilot	Python	Self-hosted completion
kiro-cli	Unknown	Broken remote

Agent	Language	Key Differentiator
conduit	Unknown	Workflow tool

Tier 5: Experimental/Minimal (15+ agents)

- AIChat, AIAgent, AIChat LLM Functions
 - Copilot CLI, Crush, DeepSeek CLI Youkpan
 - Get Shit Done, Mobile Agent, Multiagent Coding
 - Noi, Shai, Snow CLI, Superset
 - UI/UX Pro Max, Warp, X-CMD, Xela CLI
-

6. Feature Matrix

Tier 1 Feature Comparison

Feature	claude	aider	codex	openhands	cline	continue	gemini	roo	swe	vtcode
Git Integration	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗
Browser Automation	✗	✗	✗	✓	✓	✗	✗	✗	✗	✗
Sandbox/ Security	✓	✗	✓	✓	✗	✗	✗	✗	✓	✗
Voice Commands	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗
Multi-file Edit	✓	✓	✓	✓	✓	✓	✗	✓	✓	✗
Repository Mapping	✓	✓	✗	✓	✗	✓	✗	✗	✓	✗
Evaluation Framework	✗	✗	✗	✓	✗	✗	✗	✗	✓	✗
Team/ Swarm	✓	✗	✗	✓	✗	✗	✗	✗	✗	✗
Plan Mode	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗
Auto-commit	✗	✓	✓	✓	✓	✗	✗	✓	✗	✗

Feature Count by Agent

Agent	Total Features	Unique Features
claude-code-source	25	8 (KAIIROS, Dream, Teams, etc.)
aider	18	4 (Voice, Repomap, etc.)
codex	12	3 (Sandbox, Protocol)
openhands	20	5 (Evaluation, Event system)

Agent	Total Features	Unique Features
cline	14	2 (Browser automation)
continue	15	3 (Context providers)
roo-code	12	2 (Multi-file edit)
swe-agent	10	2 (SWE-bench)
gemini-cli	6	0
vtcode	4	0

7. Master Integration Plan

Phase 1: Foundation (COMPLETE)

Already implemented in HelixAgent: 1.  **Tool System** - 30+ tools from CLI agents 2.  **Permission System** - 4-layer architecture 3.  **Plan Mode** - Multi-step planning with verification 4.  **Team Management** - Agent swarms with consensus 5.  **KAIROS Service** - Always-on background assistant 6.  **Dream System** - Memory consolidation with 3-gate triggers 7.  **SubAgent System** - Full lifecycle management 8.  **Comprehensive Tests** - All passing

Phase 2: Advanced Features (Next 2-4 weeks)

2.1 Repository Mapping (From Aider) - CRITICAL

- Tree-sitter integration for code parsing
- Repository structure analysis
- Intelligent file context selection
- Language detection (100+ languages)
- **Files:** `internal/tools/repomap/repomap.go`, `parser.go`, `symbols.go`

2.2 Sandboxed Execution (From Codex) - CRITICAL

- Container-based sandboxing
- Seatbelt integration (macOS)
- Network isolation
- Resource limits
- Audit logging
- **Files:** `internal/tools/sandbox/sandbox.go`, `docker.go`, `seatbelt.go`

2.3 Browser Automation (From Cline/OpenHands) - HIGH

- Headless browser control
- Screenshot capture
- DOM interaction
- Web navigation
- Content extraction
- **Files:** `internal/tools/browser/browser.go`, `actions.go`

2.4 Edit Block Format (From Aider/Codex) - HIGH

- Search/replace blocks
- Surgical code modifications
- Minimal diff generation
- Multi-file editing
- **Files:** `internal/tools/editblock/editblock.go, parser.go`

Phase 3: Enhanced Features (Weeks 4-6)

3.1 Voice Commands (From Aider) - MEDIUM

- Speech recognition
- Voice command mapping
- Hands-free coding
- **Files:** `internal/agents/voice/voice.go`

3.2 Auto-Commit Workflow (From Aider/Codex) - MEDIUM

- Automatic commit generation
- Commit message generation from diff
- **Files:** `internal/tools/git/autocommit.go`

3.3 Evaluation Framework (From OpenHands/SWE-agent) - MEDIUM

- SWE-bench integration
- Custom benchmark metrics
- Performance evaluation pipeline
- **Files:** `internal/benchmark/evaluator.go`

3.4 Context Providers (From Continue) - MEDIUM

- Modular context system (`@file`, `@url`, `@docs`)
- Extensible provider interface
- **Files:** `internal/context/providers/`

Phase 4: Integration (Weeks 6-8)

4.1 Multi-Agent Swarm (From Claude Code) - MEDIUM

- XML-based communication
- Shared scratchpad
- Color assignment
- Role-based agents

4.2 Agent Modes (From Roo Code) - LOW

- Code mode
- Architect mode
- Ask mode
- Debug mode

- **Files:** `internal/agents/modes/modes.go`

4.3 YOLO Classifier (From Claude Code) - LOW

- ML-based auto-approval system for tool execution

Implementation Schedule

Week	Feature	Priority
1	Repository Mapping	CRITICAL
1-2	Sandboxed Execution	CRITICAL
2	Browser Automation	HIGH
2-3	Edit Block Format	HIGH
3	Voice Commands	MEDIUM
4	Auto-Commit	MEDIUM
5	Evaluation Framework	MEDIUM
6	Context Providers	MEDIUM
7-8	Integration & Polish	LOW

8. Porting Priorities

CRITICAL (Implement First)

1. **Repository Mapping** (Aider) - Tree-sitter based codebase analysis
2. **Sandboxed Execution** (Codex) - Secure command isolation

HIGH (Next Priority)

1. **Browser Automation** (Cline/OpenHands) - Headless browser for research
2. **Edit Block Format** (Aider/Codex) - Surgical code modifications
3. **Multi-file Editing** (Roo Code) - Simultaneous file changes

MEDIUM (Future)

1. **Voice Commands** (Aider) - Speech-to-code
2. **Auto-Commit Workflow** (Aider/Codex) - Automatic git operations
3. **Evaluation Framework** (OpenHands/SWE-agent) - Benchmark integration
4. **Context Providers** (Continue) - Modular context system
5. **Agent Modes** (Roo Code) - Specialized behaviors

LOW (Nice-to-Have)

1. **YOLO Classifier** (Claude Code) - ML auto-approval
 2. **Plugin Architecture** (Claude Plugins) - Extensibility framework
 3. **IDE Integration** (Continue/Junie) - Editor plugins
-

9. Key Innovations & Genial Hacks

9.1 Claude Code Source (Internal - 513K LOC)

- **KAIROS**: Always-on background assistant that observes and helps without prompt
- **Dream System**: Memory consolidation with 3-gate triggers (time, activity, memory pressure)
- **Team Management**: Multi-agent swarms with consensus algorithms
- **Plan Mode**: Multi-step planning with verification gates
- **YOLO Classifier**: ML-based auto-approval for tool execution
- **Permission System**: 4-layer permission architecture (deny, ask, yolo, full)

9.2 Aider (20K LOC)

- **Edit Block Format**: Search/replace blocks for surgical modifications
- **Repository Mapping**: Tree-sitter integration for 100+ languages
- **Git-Native Workflow**: Automatic commits with sensible messages
- **Voice-to-Code**: Full speech recognition integration
- **Lint/Test Auto-Fix Loops**: Iterative error fixing
- **88% Self-Written**: The agent writes its own code

9.3 Codex (TS/Rust)

- **macOS Seatbelt Sandboxing**: Uses `/usr/bin/sandbox-exec` for native sandboxing
- **JSON-RPC Lite Protocol**: Structured communication protocol
- **ratatui TUI**: Rich terminal UI in Rust
- **Multi-Modal Approval**: Different approval levels for different operations

9.4 OpenHands (50K LOC)

- **Event-Driven Architecture**: Action-observation loop with streamable events
- **Docker Runtime**: Container-based sandboxing with E2B integration
- **SWE-bench 77.6%**: State-of-the-art benchmark performance
- **Theory of Mind Module**: Research-grade agent cognition

9.5 Cline (15K LOC)

- **Autonomous Browser Navigation**: Self-directed web research
- **Context-Aware File Selection**: Gitignore-aware intelligent selection
- **VS Code Extension + CLI**: Dual interface architecture

9.6 Continue (30K LOC)

- **@Provider System**: Modular context with `@file`, `@url`, `@docs`, `@codebase`
- **Universal IDE**: Single codebase for VS Code, JetBrains, and LSP editors
- **Slash Action Framework**: Extensible `/command` system

9.7 SWE-agent (10K LOC)

- **Thought-Action Loop**: Explicit reasoning before action execution
- **Demonstration Trajectories**: Few-shot learning from human demonstrations
- **Computer Interface**: Unified filesystem + terminal abstraction

10. Submodule Repository Map

Tier 1 Submodules

Submodule	Repository
claude-code-source	Anthropic internal
aider	https://github.com/Aider-AI/aider
codex	https://github.com/openai/codex
openhands	https://github.com/All-Hands-AI/OpenHands
cline	https://github.com/cline/cline
continue	https://github.com/continuedev/continue
gemini-cli	https://github.com/google-gemini/gemini-cli
roo-code	https://github.com/RooVetGit/Roo-Code
swe-agent	https://github.com/princeton-nlp/SWE-agent
vtcode	https://github.com/vinhnx/vtcode

Tier 2 Submodules

Submodule	Repository
gptme	https://github.com/ErikBjare/gptme
gpt-engineer	https://github.com/AntonOsika/gpt-engineer
kilo-code	https://github.com/Kilo-Org/kilocode
junie	https://github.com/JetBrains/junie
forge	https://github.com/antinomyhq/forge
deepseek-cli	https://github.com/holasoymalva/deepseek-cli
agent-deck	https://github.com/asheshgoplani/agent-deck
zeroshot	https://github.com/supercrafter100/zeroshot
spec-kit	https://github.com/github/spec-kit

Tier 3 Submodules

Submodule	Repository
amazon-q	https://github.com/aws/amazon-q-developer-cli
claude-plugins	(plugin system)
claude-squad	(multi-agent)
open-interpreter	https://github.com/KillianLucas/open-interpreter
plandex	https://github.com/plandex-ai/plandex

Submodule	Repository
ollama-code	https://github.com/tcsenpai/ollama-code
qwen-code	https://github.com/QwenLM/qwen-code
codai	https://github.com/meysamhadeli/codai
codename-goose	https://github.com/jgeneral/codename-goose
codex-skills	https://github.com/openai/codex-skills

Tier 4 Submodules

Submodule	Repository
git-mcp	(Git MCP server)
postgres-mcp	(PostgreSQL MCP)
mistral-code	(Mistral AI)
octogen	(code gen)
nanocoder	(lightweight)
taskweaver	https://github.com/microsoft/TaskWeaver
opencode-cli	https://github.com/opencode-ai/opencode
fauxpilot	(Copilot alternative)
kiro-cli	(broken remote)
conduit	(workflow)

Tier 5 Submodules

Submodule	Repository
aichat	(chat agent)
aiagent	(generic agent)
aichat-llm-functions	(LLM functions)
copilot-cli	https://github.com/github/copilot-cli
crush	(unknown)
deepseek-cli-youkpan	(alt DeepSeek)
get-shit-done	(productivity)
mobile-agent	(mobile)
multiagent-coding	(multi-agent)
noi	(unknown)
shai	(unknown)
snow-cli	(unknown)
superset	(unknown)

Submodule	Repository
ui-ux-pro-max	(UI/UX)
warp	(terminal)
x-cmd	(command)
xela-cli	(unknown)

Special Submodules

Submodule	Notes
HelixCode	HelixAgent's own code (0 bytes - submodule)

Appendix A: Analysis Document Index

Document	Size	Agent	Tier
AIDER_ANALYSIS.md	4,076 bytes	aider	1
CLINE_ANALYSIS.md	2,188 bytes	cline	1
CODEX_ANALYSIS.md	2,780 bytes	codex	1
CONTINUE_ANALYSIS.md	1,963 bytes	continue	1
GEMINI_CLI_ANALYSIS.md	1,763 bytes	gemini-cli	1
OPENHANDS_ANALYSIS.md	3,174 bytes	openhands	1
ROO_CODE_ANALYSIS.md	1,847 bytes	roo-code	1
SWE_AGENT_ANALYSIS.md	1,938 bytes	swe-agent	1
VTCODE_ANALYSIS.md	1,279 bytes	vtcode	1
GPTME_ANALYSIS.md	531 bytes	gptme	2
GPT_ENGINEER_ANALYSIS.md	525 bytes	gpt-engineer	2
KILO_CODE_ANALYSIS.md	530 bytes	kilo-code	2
JUNIE_ANALYSIS.md	509 bytes	junie	2
FORGE_ANALYSIS.md	470 bytes	forge	2
DEEPSEEK_CLI_ANALYSIS.md	492 bytes	deepseek-cli	2
AGENT_DECK_ANALYSIS.md	435 bytes	agent-deck	2
ZEROSHOT_ANALYSIS.md	438 bytes	zeroshot	2
SPEC_KIT_ANALYSIS.md	441 bytes	spec-kit	2
MASTER_INTEGRATION_PLAN.md	7,446 bytes	ALL	Meta
TIER_1_SUMMARY.md	5,282 bytes	Tier 1	Meta
TIER_3_4_5_ANALYSIS.md	3,110 bytes	Tier 3-5	Meta
ANALYSIS_STATUS.md	2,693 bytes	ALL	Meta

Appendix B: HelixAgent Internal Features (Already Implemented)

From the analysis, HelixAgent has already implemented Phase 1 features that originate from Claude Code Source:

1. **Tool System** - 30+ tools extracted from all analyzed agents
2. **Permission System** - 4-layer architecture (deny/ask/yolo/full)
3. **Plan Mode** - Multi-step planning with verification gates
4. **Team Management** - Multi-agent swarms with consensus
5. **KAIROS Service** - Always-on background assistant
6. **Dream System** - Memory consolidation with 3-gate triggers
7. **SubAgent System** - Full lifecycle management

End of Master Catalog